

Master : IESE – S1

Travaux Pratiques : Microcontrôleurs ARM

TP2 : PWM, ADC

Objectif :

Configuration des broches GPIO pour la :

- Configuration et utilisation d'un canal ADC (Convertisseur Analogique-Digital) pour lire une grandeur analogique.
- Génération de signaux PWM.
- Implémentation d'une application pour générer un signal PWM avec une fréquence et un rapport cyclique spécifiques.
- Configuration de UART.
- Utilisation de UART pour transmettre les données lues via ADC à un périphérique externe.

Partie 1 : Génération d'un signal PWM :

Jusqu'à présent, les exemples présentés dans le TP1 ont principalement abordé l'émission de signaux numériques via les broches numériques, ainsi que la commande d'allumage et d'extinction de LED. Ces signaux numériques sont généralement définis soit comme **RESET**, correspondant à une tension de 0V, soit comme **SET**, équivalant à la tension d'alimentation de 3.3V.

Cependant, les sorties numériques ne permettent pas de régler finement des paramètres tels que la luminosité d'une LED ou la vitesse de rotation d'un moteur. Pour obtenir ce type de contrôle, il serait nécessaire de disposer de sorties capables de générer des tensions intermédiaires entre 0V et 3.3V. Une méthode couramment utilisée est l'utilisation de Pulse Width Modulation (PWM), également connue sous le nom de modulation de largeur d'impulsion en français (MLI).

Pulse Width Modulation (PWM) est une technique largement utilisée dans le domaine de l'électronique pour moduler la largeur des impulsions d'un signal périodique. Contrairement aux signaux analogiques qui varient en continu, le PWM permet de produire des signaux numériques dont la durée de l'état haut (ou actif) est modulée pour obtenir différentes valeurs de tension moyenne.

A. Principe de fonctionnement

Le fonctionnement du PWM repose sur la variation du rapport cyclique, qui est le rapport entre la durée pendant laquelle le signal est **haut** et la **période** totale du signal. En ajustant ce rapport cyclique, on peut contrôler la quantité d'énergie transmise au système de sortie, ce qui permet de réguler divers paramètres tels que la vitesse, la luminosité, ou la puissance.

$$DutyCycle[PWM] = \frac{T_{ON}}{T_{ON} + T_{OFF}} \times 100 = \frac{T_{ON}}{Period(T)} \times 100[\%]$$

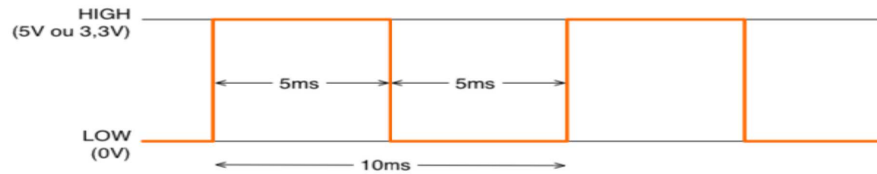


Figure 1: PWM à 50% (La fréquence est de 100Hz, le rapport cyclique de 50%)

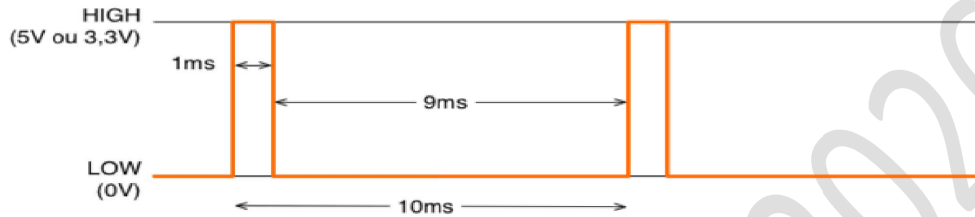


Figure 2: PWM à 10% (La fréquence est de 100Hz et le rapport cyclique de 10%)

La résolution PWM est exprimée en (bits). C'est le nombre de bits utilisés pour représenter la valeur du rapport cyclique. Il peut s'agir de 8 bits, 10, 12 ou même 16 bits. La résolution PWM peut correspondre au nombre de niveaux de rapport cyclique discrets compris entre 0 % et 100 %. Plus la résolution PWM est élevée, plus le nombre de niveaux discrets est élevé sur toute la plage du cycle de service du PWM.

Une résolution PWM de seulement **3 bits** signifie qu'il n'y a que **8 niveaux** discrets pour le rapport cyclique sur toute la plage (de 0 % à 100 %). D'un autre côté, un PWM avec une résolution de **8 bits** aura **256** niveaux discrets pour le rapport cyclique sur toute la plage (de 0 % à 100 %).

Le PWM est largement utilisé dans de nombreuses applications, notamment :

- Contrôle de la vitesse des moteurs
- Alimentation à découpage
- Contrôle de la luminosité des LED
- Régulation de la température ...

B. Génération d'un Signal PWM sur la carte STM32F429ZI

Dans le TP1, nous avons utilisé le périphérique TIM6 du STM32F429ZI, mais le TIM6 ne prend pas en charge la génération PWM. Pour générer un signal PWM, nous devons utiliser Timer polyvalente ou avancée (General purpose or Advanced control Timers).

Table 6. Timer feature comparison

Timer type	Timer	Counter resolution	Counter type	Prescaler factor	DMA request generation	Capture/compare channels	Complementary output	Max interface clock (MHz)	Max timer clock (MHz) ⁽¹⁾
Advanced-control	TIM1, TIM8	16-bit	Up, Down, Up/down	Any integer between 1 and 65536	Yes	4	Yes	90	180
General purpose	TIM2, TIM5	32-bit	Up, Down, Up/down	Any integer between 1 and 65536	Yes	4	No	45	90/180
	TIM3, TIM4	16-bit	Up, Down, Up/down	Any integer between 1 and 65536	Yes	4	No	45	90/180
	TIM9	16-bit	Up	Any integer between 1 and 65536	No	2	No	90	180
	TIM10, TIM11	16-bit	Up	Any integer between 1 and 65536	No	1	No	90	180
	TIM12	16-bit	Up	Any integer between 1 and 65536	No	2	No	45	90/180
	TIM13, TIM14	16-bit	Up	Any integer between 1 and 65536	No	1	No	45	90/180
	TIM6, TIM7	16-bit	Up	Any integer between 1 and 65536	Yes	0	No	45	90/180

1. The maximum timer clock is either 90 or 180 MHz depending on TIMPRE bit configuration in the RCC_DCKCFGR register.

Les périphériques des Timer « General purpose » et « Advanced control » sont également livrés avec un (ou plusieurs) canal(s) configurable(s). Ces canaux peuvent être utilisés pour les opérations de capture/comparaison, la sortie à une impulsion et la génération de signaux PWM. Nous sélectionnerons le périphérique **TIM4** connecté au **APB1**.

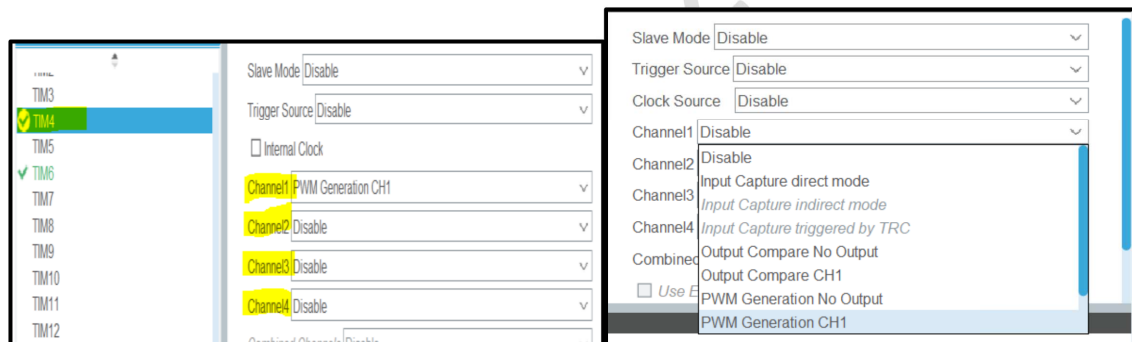


Figure 3: Configuration du Timer2

Le signal PWM est généré en fonction de la valeur du registre de capture/comparaison (**CCR1**) (Image ci-dessous). En outre, nous devons également configurer le canal et la broche appropriés pour prendre en charge cette fonctionnalité. Lors de l'utilisation de la fonction **PWM**, le **PSC** et l'**ARR** définissent la fréquence du signal ainsi que la fréquence de la source d'horloge de la minuterie. Le **CCR_x** définit le rapport cyclique du signal PWM.

Le mode PWM permet de générer un signal avec une fréquence déterminée par la valeur des registres **TIM_x_PSC** et **TIM_x_ARR**, et un rapport cyclique déterminé par la valeur du registre **TIM_x_CCR_x**.

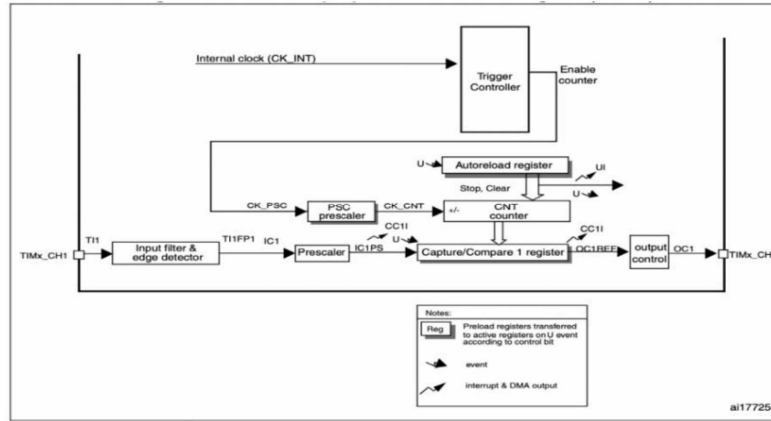


Figure 4: Diagram block du Timer avec canal PWM

La valeur du registre **CCR_x** doit être supérieure à 0 et **inférieure** ou égale à la valeur de **ARR**. Sinon, le signal PWM n'aura pas de rapport cyclique en pourcentage. Lorsque le Timer est activée et configurée pour la génération de signal PWM, la valeur du compteur augmente (ou diminue s'il s'agit d'un compte à rebours (counting down)) et la valeur du compteur se réinitialise lorsqu'elle atteint la valeur **ARR**. Le signal de sortie bascule lorsque la valeur **CNT** atteint la valeur **CCR_x**. Et il revient à son état initial lorsque la valeur du compteur est réinitialisée.

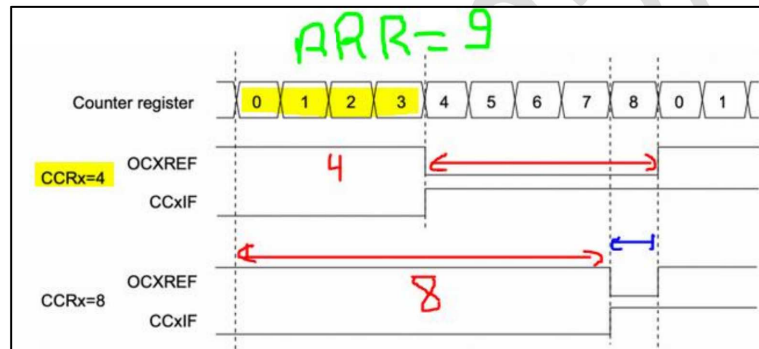


Figure 5: Exemple de registre CCR pour les valeurs 4 et 8 avec an ARR = 9

Nous pouvons écrire n'importe quelle valeur entre 0 et 65 535 dans le **CCR_x** mais pour générer un signal PWM, elle doit être $0 < \text{CCR}_1 \leq \text{ARR}$. Si le **CCR_x** est 0, le rapport cyclique est de 0 % et s'il est supérieur à la valeur **ARR**, le rapport cyclique est de 100 %.

$$\text{TIM_CLK} = \frac{\text{APB_CLK}}{\text{TIM_PSC}}, \quad \text{Frequence} = \frac{\text{TIM_CLK}}{\text{TIM_ARR}}, \quad \text{duty_cycle} = \frac{\text{CCR}_x}{\text{TIM_ARR}} \times 100$$

NB : Notez que nous avons sélectionné le canal 1, donc **CCR_x = CCR1**.

En utilisant la même configuration pour le **TIMER6 (TP1)** avec un **PSC** de 8999 et un **ARR** de 4999 pour générer une fréquence de **10 kHz** (période = 0.1 ms). Bien que la valeur de **CCR1** puisse être ajustée dans la configuration "**Pulse (16 bits value)**", nous la modifierons dans le code pour obtenir différents rapports cycliques.

auto-reload preload	Enable
Trigger Output (TRGO) Parameters	
Master/Slave Mode (MSM bit)	Disable (Trigger input effect not delayed)
Trigger Event Selection	Reset (UG bit from TIMx_EGR)
PWM Generation Channel 1	
Mode	PWM mode 1
Pulse (16 bits value)	
Output compare preload	Enable
Fast Mode	Disable
CH Polarity	High

Après avoir généré le code de configuration, il est nécessaire d'appeler la fonction suivante pour lancer le **TIMER4** et commencer la génération du signal PWM sur le canal 1.

HAL_TIM_PWM_Start(&htim4, TIM_CHANNEL_1);

Ensuite pour changer la valeur du rapport cyclique, soit on utilise le registre CCR1 directement

TIM4->CCR1 = Dc_Value;

Ou on utilise la fonction où **Dc_Value** est la valeur du CCR1 :

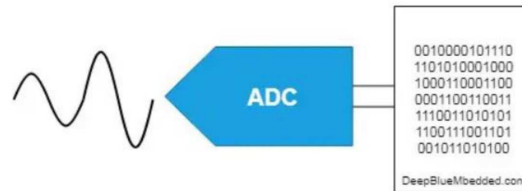
HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_1, Dc_Value);

Par exemple si Dc_Value=2500 cela génère PWM avec un rapport cyclique de 50%.

Manipulation : Produire un signal PWM avec une fréquence de 5 kHz et ajuster le rapport cyclique entre 10% et 100% pour moduler la luminosité d'une LED. Observer simultanément le signal PWM à l'oscilloscope pour vérifier la configuration (*Enregistrer les signaux récupérés pour quelques valeurs du rapport cyclique*).

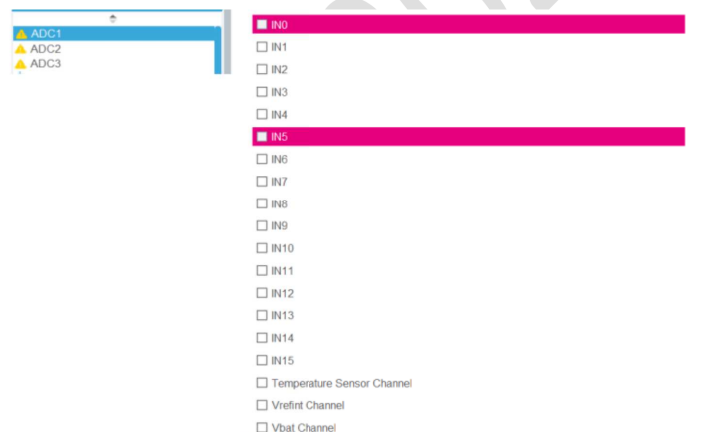
Partie 2 : Le convertisseur Analogique Numérique (ADC) :

Un convertisseur ADC (Analog-to-Digital Converter) est un circuit électronique qui prend une tension analogique en entrée et la convertit en données numériques, une valeur qui représente le niveau de tension en code binaire. L'ADC échantillonne l'entrée analogique chaque fois que vous le déclenchez pour démarrer la conversion. Il effectue un processus appelé quantification pour décider du niveau de tension et de son code binaire qui est inséré dans le registre de sortie.



A. ADC in STM32 Microcontrollers

Le STM32F446Re dispose de trois ADC 12 bits. Il dispose de jusqu'à 24 canaux multiplexés lui permettant de mesurer des signaux provenant de seize sources externes et de deux sources internes. La conversion A/D des différents canaux peut être effectuée en mode simple, continu, balayage ou discontinu. Le résultat de l'ADC est stocké dans un registre de données de 16 bits aligné à gauche ou à droite.



☞ Alignement des données des résultats ADC

Les données de résultat ADC ont une largeur de 12 bits et le registre tampon est de 16 bits. Par conséquent, il peut être aligné à gauche ou à droite, comme indiqué dans le diagramme ci-dessous. La valeur des données converties des canaux de groupe injectés peut être diminuée du décalage défini par l'utilisateur écrit dans les registres ADC_JOFRx.



☞ Modes de fonctionnement du ADC en STM32

- 1- **Mode de conversion unique (single conversion mode) :** En mode Conversion unique, l'ADC effectue une conversion. Une fois la conversion de la chaîne sélectionnée terminée :

Si une chaîne normale a été convertie :

- ☞ Les données converties sont stockées dans le registre ADC_DR 16 bits
- ☞ Le drapeau EOC (End Of Conversion) est activé
- ☞ et une interruption est générée si l'EOCIE est activé.
- ☞ *L'ADC est alors arrêté.*

- 2- **Mode de conversion continu (Continuous Conversion Mode) :** En mode de conversion continue, ADC démarre une autre conversion dès qu'il en termine une. Après chaque conversion, il effectue les mêmes étapes utilisées en mode de conversion unique mais sans arrêter l'ADC.

- 3- **Mode de balayage (scan mode) :** Ce mode est utilisé pour scanner un **groupe** de chaînes analogiques. Une seule conversion est effectuée pour chaque canal du groupe. Après chaque fin de conversion, la chaîne suivante du groupe est automatiquement convertie.

Lors de l'utilisation du mode scan, le bit DMA doit être défini et le contrôleur d'accès direct à la mémoire est utilisé pour transférer les données converties des canaux de groupe réguliers vers SRAM après chaque mise à jour du registre ADC_DR.

- 4- **Mode discontinu (Discontinuous Mode) :** Ce mode est activé en définissant le bit DISCEN dans le registre ADC_CR1. Il peut être utilisé pour convertir une courte séquence de **n conversions** ($n \leq 8$) qui fait partie de la séquence de conversions sélectionnée dans les registres ADC_SQRx

- **Temps d'échantillonnage (STM32 ADC Sampling Time) :** L'ADC STM32 échantillonne la tension d'entrée pendant un certain nombre de cycles ADC_CLK

Le temps total de conversion ADC est calculé comme suit :

$$T_{\text{conv}} = \text{Temps d'échantillonnage} + 12,5 \text{ cycles}$$

Exemple: Avec un ADCCLK = 14 MHz et un temps d'échantillonnage de 1,5 cycles :

$$T_{\text{conv}} = 1,5 + 12,5 = 14 \text{ cycles} = 1 \mu\text{s}$$

- **Tension de référence ADC (ADC Reference Voltage) :**

Les broches de tension de référence ADC sont définies dans la fiche technique et supposées être connectées à un niveau de tension dans une certaine plage. Ceci est indiqué dans le tableau ci-dessous. Vous pouvez choisir de régler la tension de référence à son niveau maximum ou minimal autorisé.

Name	Signal type	Remarks
V _{REF+}	Input, analog reference positive	The higher/positive reference voltage for the ADC, $2.4 \text{ V} \leq V_{\text{REF+}} \leq V_{\text{DDA}}$
V _{DDA} ⁽¹⁾	Input, analog supply	Analog power supply equal to V _{DD} and $2.4 \text{ V} \leq V_{\text{DDA}} \leq 3.6 \text{ V}$
V _{REF-}	Input, analog reference negative	The lower/negative reference voltage for the ADC, $V_{\text{REF-}} = V_{\text{SSA}}$
V _{SSA} ⁽¹⁾	Input, analog supply ground	Ground for analog power supply equal to V _{SS}

Vous devrez consulter le schéma de carte de développement schéma car il se peut qu'elle ne connecte pas du tout V_{ref} ou il est connecté à 2,5 V par exemple, donc l'ADC saturera et vous donnera 4096 (2¹⁶) avant la tension analogique d'entrée atteint 3,3 V et vous vous demandez pourquoi ! c'est peut-être parce que la tension de référence est réglée sur une valeur inférieure à 3,3 V, c'est donc quelque chose à considérer.

- **STM32 ADC Formulas :**

ADC Conversion Time

$$T_{\text{conv}} = \text{Sampling time} + 12.5 \text{ cycles}$$

ADC Sampling Rate

$$\text{SamplingRate} = 1 / T_{\text{conv}}$$

ADC Result Voltage (Analog Input Value)

$$V_{\text{in}} = \text{ADC_val} \times (\text{Reference Voltage} / 4096)$$

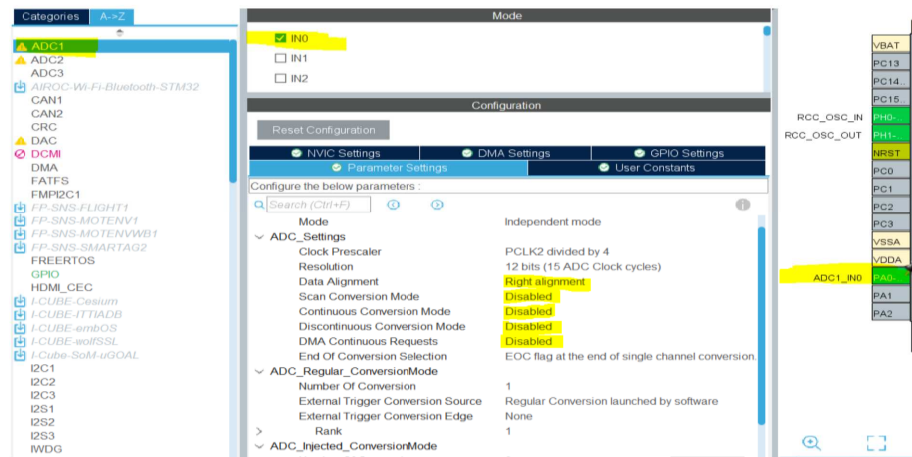
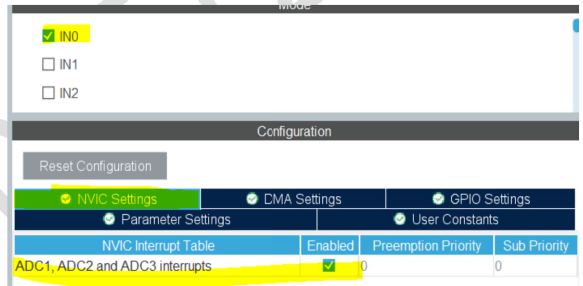


Figure 9: Configuration du ADC1_IN0

2. Single conversion (Interrupt method) :

Utilisez la même configuration que l'exemple précédent. Tous les paramètres ADC resteront les mêmes mais nous devons activer l'interruption depuis l'onglet du contrôleur NVIC.

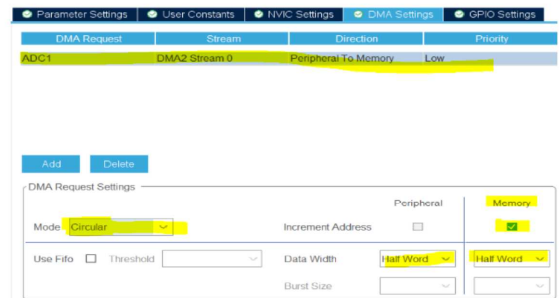
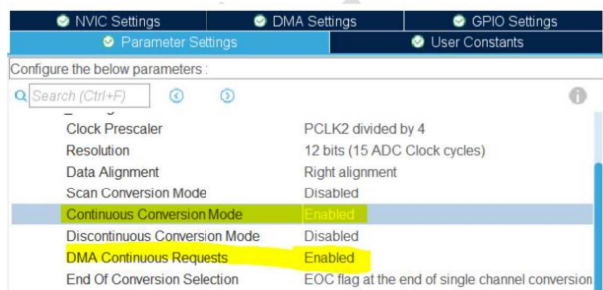
Transférez la valeur de l'ADC vers l'UART ainsi que la valeur de tension mesurée.



3. Single conversion (DMA method) :

Tout dans les configurations ADC sera par défaut en mode normal. Cependant, cette fois, les interruptions STM32 ADC ne sont pas activées, le DMA est configuré et l'interruption DMA est activée par défaut dans l'onglet du contrôleur NVIC. Les configurations STM32 DMA pour l'ADC seront telles qu'indiquées ci-dessous.

Transférez la valeur de l'ADC (récupérer depuis le DMA) vers l'UART ainsi que la valeur de tension mesurée.



Fonctions utiles à utiliser :

- Calibrer l'ADC à la mise sous tension pour une meilleure précision

```
if (HAL_ADCEx_Calibration_Start(&hadc1) != HAL_OK) {  
Error_Handler(); // Calibration error  
}
```

- Démarrer le ADC

```
HAL_ADC_Start(&hadc1); // généralement appelée dans la fonction 'main()'
```

- Interroger (Poll) le périphérique ADC1 et le délai d'attente = 1 mSec

```
HAL_ADC_PollForConversion(&hadc1, 1);
```

- Lire le résultat de la conversion ADC

```
uint16_t adc_value = 0; // à déclarer dans la fonction main  
float VRefp = 3.3; // Supposant VREF+ est connecté à 3.3V  
float Vin = 0; //  
adc_value = HAL_ADC_GetValue(&hadc1);  
Vin = adc_value * (VRefp / 4096);
```

- Démarrer l'ADC en mode de conversion en interruption

```
HAL_ADC_Start_IT(&hadc1);  
  
//cette fonction de Callback doit être définie dans la partie /*user code 4  
  
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)  
{  
    // Lecture et la mise à jour du résultat de l'ADC  
    adc_value = HAL_ADC_GetValue(&hadc1);  
}
```

- Démarrer l'ADC en mode de conversion en DMA

```
uint16_t adc_value; // Déclaration de la mémoire tampon (half-word)  
HAL_ADC_Start_DMA(&hadc1, (uint32_t *) &adc_value, 1);  
// La variable adc_value stock la valeur récupérée de l'ADC.
```



```
uint32_t ADC_ReadOnce_Polling(void)
{
    uint32_t val = 0;

    HAL_ADC_Start(&hadc1);

    if (HAL_ADC_PollForConversion(&hadc1, 10) == HAL_OK) // timeout ms
    {
        val = HAL_ADC_GetValue(&hadc1);
    }

    HAL_ADC_Stop(&hadc1);
    return val;
}
```

$t_{conv} \approx (56 + 12.5) / 22.5 \text{ MHz} \approx 3.0 \mu\text{s}$